西安交通大學
XIAN JIAOTONG UNIVERSITY

现代密码学 COMP401227

第 3 章：流密码

赵俊舟

西安交通大学网安学院
junzhou.zhao@xjtu.edu.cn

2026 年 4 月 23 日

目录

1. 基本概念
2. 密钥流生成器
3. 几种实现

目录

1 基本概念
- 同步流密码
- 有限状态自动机

2 密钥流生成器

3 几种实现

目录

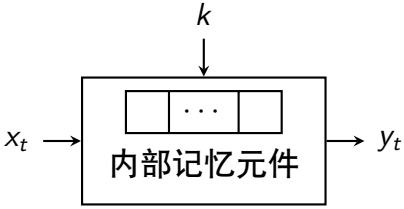
1 基本概念
- 同步流密码
- 有限状态自动机

2 密钥流生成器

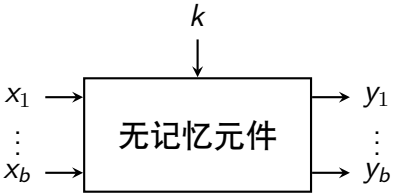
3 几种实现

流密码

- 在电话语音通信、TCP/UDP 通信等场景下，会产生持续的语音或数据等**流式数据**。
- 对这类数据流进行加密需要设计**流密码**（Stream Cipher），或**序列密码**（Sequential Cipher）。
- 流密码对明文消息按比特或字节逐位加密，不同于分组密码按定长数据块进行加密。



流密码

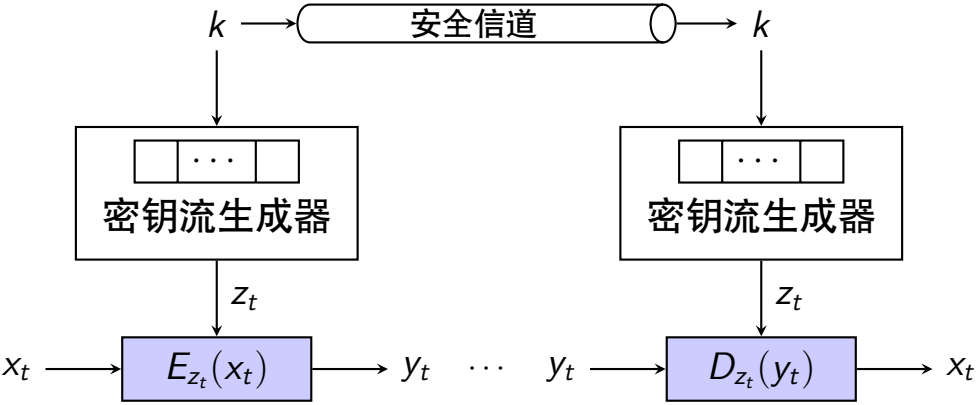


分组密码

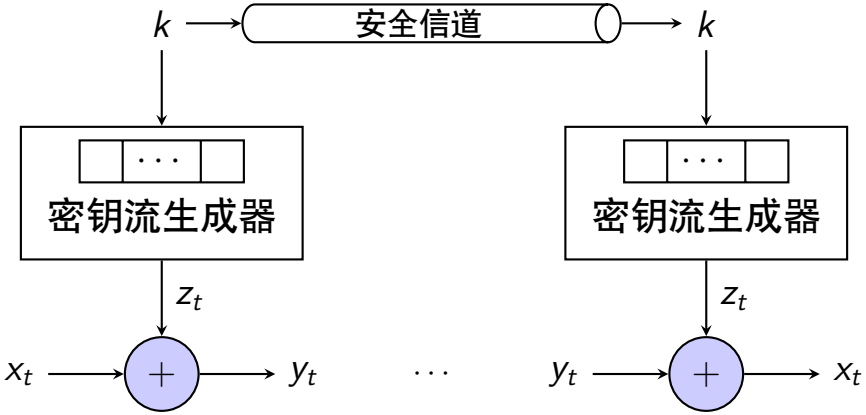
流密码的基本思想

- 利用密钥 k 产生密钥流 $z = z_0 z_1 z_2 \cdots$
- 对明文串 $x = x_0 x_1 x_2 \cdots$ 加密得到密文串 $y = y_0 y_1 y_2 \cdots$，其中
$$y_t = E_{z_t}(x_t)$$
- 密钥流 z 由密钥流发生器 f 产生：
$$z_t = f(k, \sigma_t)$$
其中 σ_t 是内部记忆元件在时刻 t 时的状态。
- 流密码具有记忆性，其内部记忆元件由一组移位寄存器构成。
- 同步流密码：内部状态与明文无关的流密码。
- 目前对同步流密码的研究比较深入。

同步流密码体制模型



加法同步流密码体制模型



加密变换为 $y_t = z_t \oplus x_t$，其原型为一次一密。

目录

1 基本概念
- 同步流密码
- 有限状态自动机

2 密钥流生成器

3 几种实现

有限状态自动机模型（Finite State Automata）

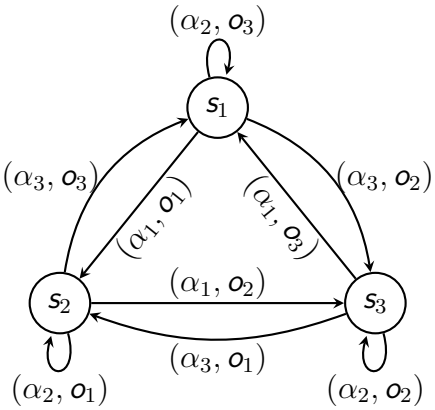
定义（有限状态自动机）

具有有限离散输入、输出，包含以下部分：

1. 有限状态集 $S = \{s_i : i = 1, 2, \ldots, r\}$；
2. 有限输入字符集 $I = \{\alpha_j : j = 1, 2, \ldots, m\}$ 和有限输出字符集 $O = \{o_k : k = 1, 2, \ldots, n\}$；
3. 输出函数
$$o = \psi(s, \alpha)$$
即在状态 s 输入为 α 时，输出 o。
4. 状态转移函数
$$s' = \phi(s, \alpha)$$
即在状态 s 输入为 α 时，状态转移到 s'。

有限状态自动机的有向图表示

- 有限状态自动机可用有向图表示，称为转移图。
- 转移图的顶点对应于有限状态自动机的状态。
- 转移图的边上标有输入输出字符。



有限状态自动机的矩阵表示

设 $S = \{s_1, s_2, s_3\}$， $I = \{\alpha_1, \alpha_2, \alpha_3\}$， $O = \{o_1, o_2, o_3\}$

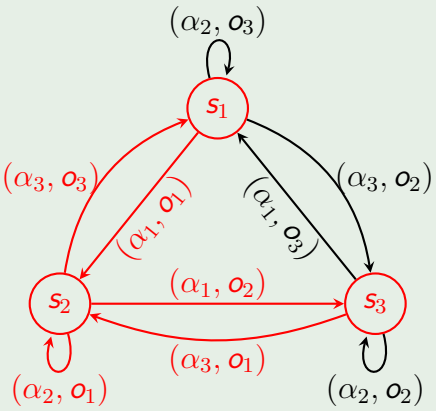
ψ	α_1	α_2	α_3
s_1	o_1	o_3	o_2
s_2	o_2	o_1	o_3
s_3	o_3	o_2	o_1

ϕ	α_1	α_2	α_3
s_1	s_2	s_1	s_3
s_2	s_3	s_2	s_1
s_3	s_1	s_3	s_2

举例：有限状态自动机

例（有限状态自动机）

- 输入序列：$\alpha_1\alpha_2\alpha_1\alpha_3\alpha_3\alpha_1$
- 状态序列为：$s_1s_2s_2s_3s_2s_1s_2$
- 输出字符序列：$o_1o_1o_2o_1o_3o_1$



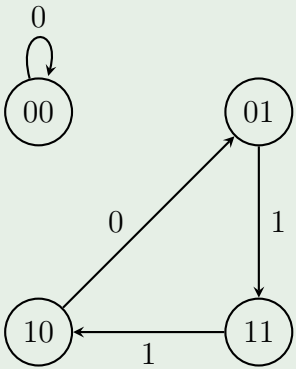
举例：有限状态自动机

例 (周期序列)

设 $S = \{00, 01, 10, 11\}$，$I = \emptyset$，$O = \{0, 1\}$，转移图如图所示。

- 若初始状态为 00
- 则状态序列为：
  $00, 00, 00, 00, \ldots$
- 输出字符序列为：$0000\cdots$

- 若初始状态为 01
- 则状态序列为：
  $01, 11, 10, 01, \ldots$
- 输出字符序列为：$1101\cdots$



两种情况下输出序列都为周期序列，周期分别为 1 和 3。

目录

1 基本概念

2 密钥流生成器
- 基本模型
- 线性反馈移位寄存器
- 非线性序列

3 几种实现

目录

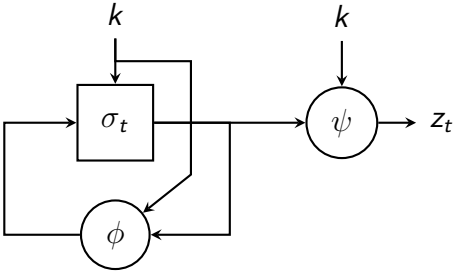
1 基本概念

2 密钥流生成器
- 基本模型
- 线性反馈移位寄存器
- 非线性序列

3 几种实现

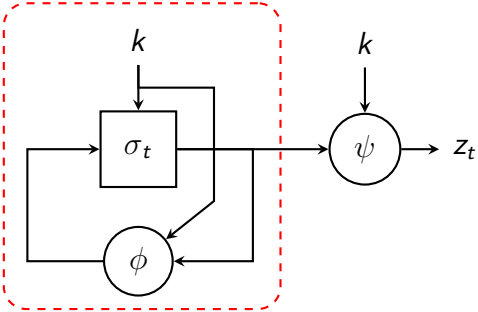
密钥流生成器的一般模型

- 可以看作是一个参数为密钥 k，没有输入字符集的有限状态自动机。
- 包含输出符号集 Z，状态集 Σ，两个函数 ϕ、ψ，以及初始状态 σ_0。
  - 状态转移函数 $\phi\colon \Sigma \mapsto \Sigma$ 将当前状态 σ_t 变为新状态 σ_{t+1}
  - 输出函数 $\psi\colon \Sigma \mapsto Z$ 将当前状态 σ_t 变为输出符号 z_t



密钥流生成器设计

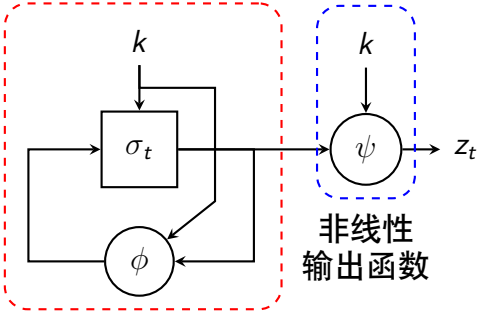
- 目标：设计状态转移函数 ϕ 和输出函数 ψ，使得输出序列 z 满足随机性要求，并且易于分析和实现。
- 方式一：非线性反馈移位寄存器 + 简单的输出函数



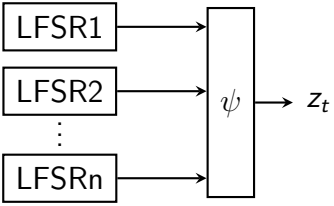
非线性反馈移位寄存器

密钥流生成器设计

- 方式二：线性反馈移位寄存器 + 非线性输出函数
- 方式三：多个线性反馈移位寄存器 + 非线性输出函数



线性反馈移位寄存器
(LFSR)



- 方式二和方式三比方式一更易于分析和实现，是目前最为流行和实用的密钥流生成器工作方式。

目录

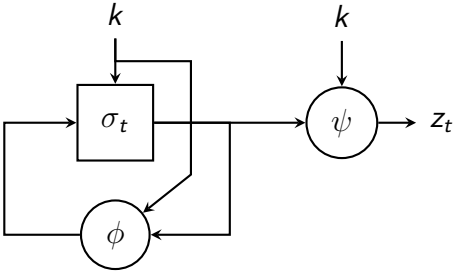
1 基本概念

2 密钥流生成器
- 基本模型
- 线性反馈移位寄存器
- 非线性序列

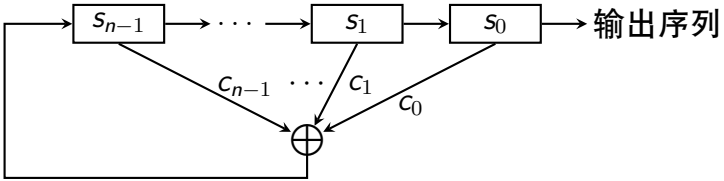
3 几种实现

密钥流生成器的一般模型

- 可以看作是一个参数为密钥 k，没有输入字符集的有限状态自动机。
- 包含输出符号集 Z，状态集 Σ，两个函数 ϕ、ψ，以及初始状态 σ_0。
  - 状态转移函数 $\phi\colon \Sigma \mapsto \Sigma$ 将当前状态 σ_t 变为新状态 σ_{t+1}
  - 输出函数 $\psi\colon \Sigma \mapsto Z$ 将当前状态 σ_t 变为输出符号 z_t



线性反馈移位寄存器 (Linear Feedback Shift Register, LFSR)



- 一个 n 级线性反馈移位寄存器包含 n 个寄存器 $s_{n-1}, \ldots, s_0$，构成当前状态 $s_{n-1} \cdots s_0$。
- 反馈系数 $c_i \in \{0, 1\}$，状态更新满足递推关系

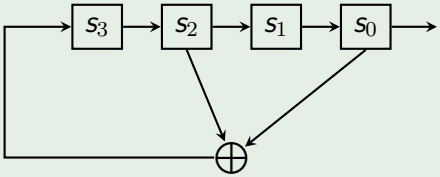
$$s_i^{(t+1)} = s_{i+1}^{(t)}, \quad i = 0, \ldots, n-2 \qquad s_{n-1}^{(t+1)} = \bigoplus_{i=0}^{n-1} c_i s_i^{(t)}$$

- 每个时钟输出最右侧寄存器的值。
- LFSR 实现简单、速度快、理论成熟。

LFSR 举例

例 (四级线性反馈移位寄存器)

初始状态 $(s_3, s_2, s_1, s_0) = 0011$



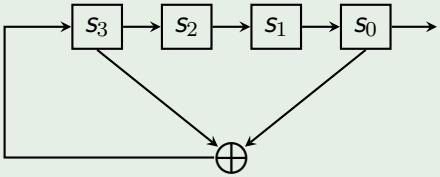
(0,0,1,1)
(1,0,0,1)
(1,1,0,0)
(1,1,1,0)
(1,1,1,1)
(0,1,1,1)
(0,0,1,1) ← 开始重复

- 输出: 110011110...
- 周期: 6

LFSR 举例

例 (四级线性反馈移位寄存器)

初始状态 $(s_3, s_2, s_1, s_0) = 0011$



(0,0,1,1)	(1,1,1,1)
(1,0,0,1)	(0,1,1,1)
(0,1,0,0)	(1,0,1,1)
(0,0,1,0)	(0,1,0,1)
(0,0,0,1)	(1,0,1,0)
(1,0,0,0)	(1,1,0,1)
(1,1,0,0)	(0,1,1,0)
(1,1,1,0)	(0,0,1,1) ← 开始重复

- 输出: 11001000111101011...
- 周期: 15

LFSR 的性质

- LFSR 输出序列的性质完全由反馈系数 c_i 确定。
- 假定 c_i 中至少有一个不为 0，否则输出恒为 0。
- n 级 LFSR 的寄存器值 $s_{n-1} \cdots s_0$ 构成其状态，共 2^n 个状态。
- n 级 LFSR 的状态周期小于等于 $2^n - 1$。
- 输出序列的周期等于状态周期，也小于等于 $2^n - 1$。
- LFSR 输出序列的周期由反馈系数 c_i 确定，选择合适的系数可使序列的周期达到最大值 $2^n - 1$。

LFSR 的安全性

- 达到最大周期的 LFSR 产生的序列具有很好的统计特性：输出序列中 0 和 1 的个数近似相等。
- 但是 LFSR 产生的序列不能作为密钥流使用，有安全问题。
- 如果 LFSR 的反馈系数公开（根据 Kerckhoffs 准则），那么 LFSR 输出的前 n 个比特其实是 LFSR 的初始状态。
- 密码分析者根据初始状态和反馈系数可以完全预测 LFSR 未来的输出序列，使密钥流完全可预测。
- 如果不公开反馈系数，密码分析者可以由接下来 n 个输出构建 n 个线性方程，由线性方程组计算出反馈系数。
- 为了抵抗以上攻击，需要使输出序列非线性化。

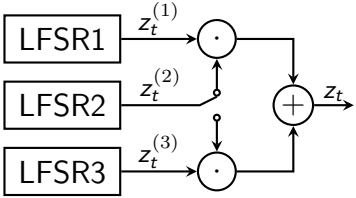
目录

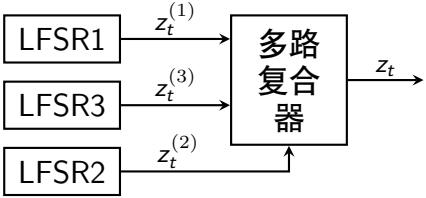
1 基本概念

2 密钥流生成器
- 基本模型
- 线性反馈移位寄存器
- 非线性序列

3 几种实现

Geffe 序列生成器

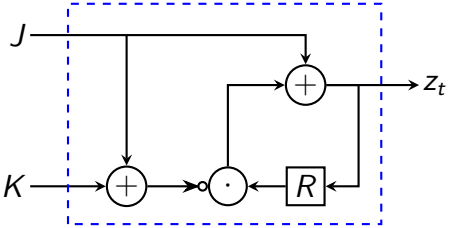




- Geffe 序列生成器由 3 个 LFSR 组成，其中 LFSR2 作为控制生成器使用。
- 当 LFSR2 输出 1 时，LFSR2 与 LFSR1 相连接；
- 当 LFSR2 输出 0 时，LFSR2 与 LFSR3 相连接；
- 输出序列可以表示为

$$z_t = z_t^{(1)} z_t^{(2)} \oplus z_t^{(3)} \overline{z_t^{(2)}} = z_t^{(1)} z_t^{(2)} \oplus z_t^{(3)} z_t^{(2)} \oplus z_t^{(3)}$$

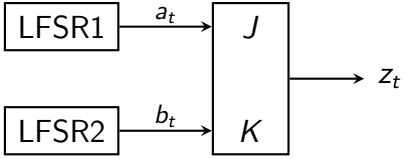
JK 触发器



x_J	x_K	z_t
0	0	z_{t-1}
0	1	0
1	0	1
1	1	$\overline{z_{t-1}}$

- JK 触发器的输出 z_t 不仅依赖输入 x_J, x_K 还依赖于上一时刻的输出 z_{t-1}

$$z_t = \overline{(x_J \oplus x_K)} z_{t-1} \oplus x_J$$

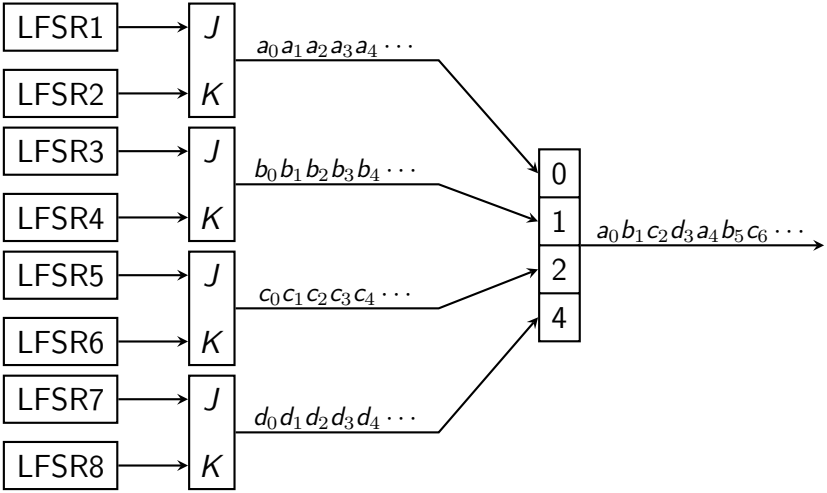


- 输出序列为

$$\begin{aligned} z_t &= \overline{(a_t \oplus b_t)} z_{t-1} \oplus a_t \\ &= (a_t \oplus b_t \oplus 1) z_{t-1} \oplus a_t \end{aligned}$$

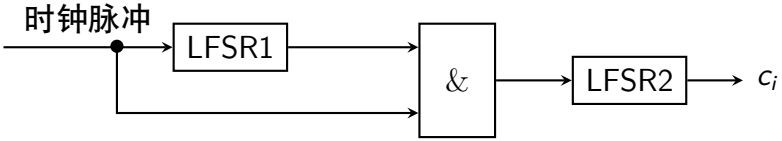
Pless 生成器

- 由 8 个 LFSR、4 个 JK 触发器和 1 个循环计数器构成。
- 由循环计数器选通控制，假定时刻 t 输出第 $t \bmod 4$ 个单元。



钟控序列生成器

- 钟控序列最基本的模型是用一个 LFSR 控制另外一个 LFSR 的移位时钟脉冲。



- 当 LFSR1 输出 1 时，移位时钟脉冲通过与门使 LFSR2 进行一次移位，从而生成下一位。
- 当 LFSR1 输出 0 时，移位时钟脉冲无法通过与门影响 LFSR2。因此 LFSR2 重复输出前一位。

目录

1 基本概念

2 密钥流生成器

3 几种实现
- RC4 流密码
- Trivium 流密码

目录

1 基本概念

2 密钥流生成器

3 几种实现
- RC4 流密码
- Trivium 流密码

RC4 算法

- RC4 算法是 Ron Rivest 在 RSA 公司于 1987 年设计的一种流密码。
- 很长一段时间内，RC4 作为 RSA 公司的商业机密并没有公开，直到 1994 年，RC4 才在匿名邮件列表上公开。
- 密钥长度可变，面向字节操作，简单高效。
- 使用广泛，包括 WEP, WPA, BitTorrent, SSH, Remote Desktop Protocol, PDF, Skype 等。
- 目前已经不安全，替代密码是 ChaCha20 流密码。

RC4 算法概述

- 一个 256 字节的状态向量 $S: S[0], \ldots, S[255]$；
- 在生成密钥流的过程中，S 反复被置换，置换后的 S 始终包含 $0 \sim 255$ 所有 8 位整数；
- 密钥流的每个字节是从 S 的 255 个元素中按照一种系统化的方法选出的一个元素生成的；
- 每产生密钥流中的一个字节，S 就被重新置换一次。

初始化 S

- S 中元素的值按升序被置为 0 到 255；
- 同时，建立一个 256 字节的临时向量 T：若输入密钥 K 长度为 256 字节，则将 K 赋给 T；否则，循环使用密钥赋值 T。

```
for i in range(256):
    S[i] = i
    T[i] = K[i mod keylen]
```

- 然后用 T 产生 S 的初始置换：对每个 $S[i]$，根据由 $T[i]$ 确定的方案，将 $S[i]$ 置换为 S 中的另一个字节。

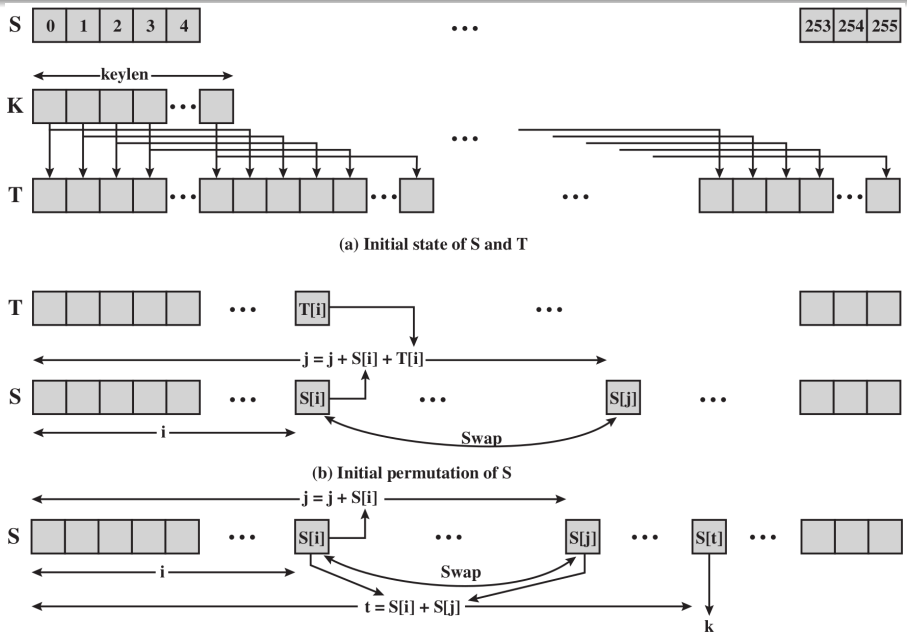
```
j = 0
for i in range(256):
    j = (j + S[i] + T[i]) mod 256
    Swap(S[i], S[j])
```

密钥流生成

- 状态向量 S 初始完后，输入密钥 K 就不再被使用。
- 从 $S[0]$ 到 $S[255]$，对每个 $S[i]$，根据 S 当前配置，将 $S[i]$ 与 S 中的另一个字节置换。
- 当 $S[255]$ 完成置换后，操作重复从 $S[0]$ 开始。

```
i, j = 0
while True:
    i = (i + 1) mod 256
    j = (j + S[i]) mod 256
    Swap(S[i], S[j])
    t = (S[i] + S[j]) mod 256
    k = S[t]
```

RC4 算法图示


S
0
1
2
3
4
253
254
255
keylen
K
T
(a) Initial state of S and T
T
T[i]
j = j + S[i] + T[i]
S
S[i]
S[j]
i
Swap
(b) Initial permutation of S
j = j + S[i]
S
S[i]
S[j]
S[t]
i
Swap
t = S[i] + S[j]
k

目录

1 基本概念

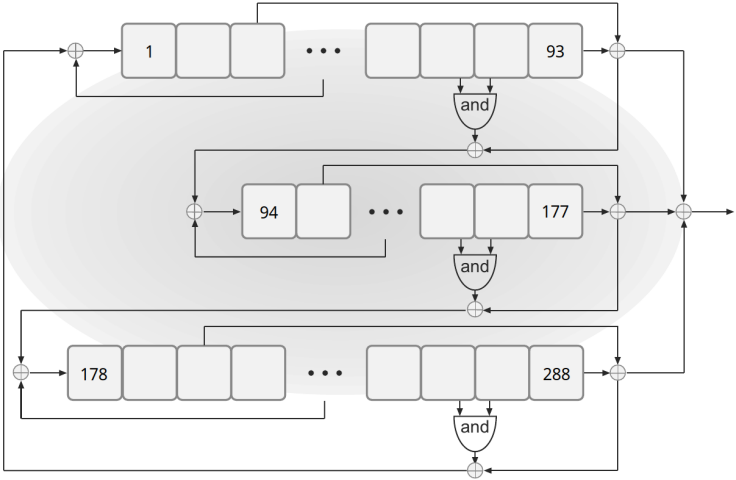
2 密钥流生成器

3 几种实现
- RC4 流密码
- Trivium 流密码

Trivium 流密码介绍

- Trivium 流密码由比利时密码学家于 2008 年提出。
- 是一种轻量级流密码，可以使用较少的门电路实现，需要较少的计算、存储等资源，适宜应用于嵌入式系统。
- 包含三组相互耦合的非线性反馈移位寄存器 A, B 和 C，级数分别为 $93, 84$ 和 111，所以 Trivium 的状态共包含 288 比特。
- 密钥长度为 80 比特，初始向量 IV 为 80 比特。
- 反馈函数：每组寄存器的部分寄存器值经过非线性运算，然后和下一组中某个寄存器值异或，作为下一组寄存器最左边的寄存器值。
- 输出函数：每组寄存器最右边的寄存器值和本组中某个寄存器值分别异或后，再异或，作为输出密钥流。
- 安全性：目前尚无比穷举攻击更好的攻击方法。

Trivium 流密码结构



- **初始化**：用 80 位密钥填充寄存器 A 的最左边 80 个寄存器，用 80 位 IV 填充寄存器 B 的最左边 80 个寄存器，寄存器 C 的最右边 3 个寄存器设为 1，其余寄存器都设为 0。

小结

1. 基本概念
2. 密钥流生成器
3. 几种实现